Summary of Pointers

Summary

- Basic Concepts
- Pointers and Arrays
- Pointers to pointers
- Pointers and Functions
- Dynamic Memory Allocation

Basic Concepts

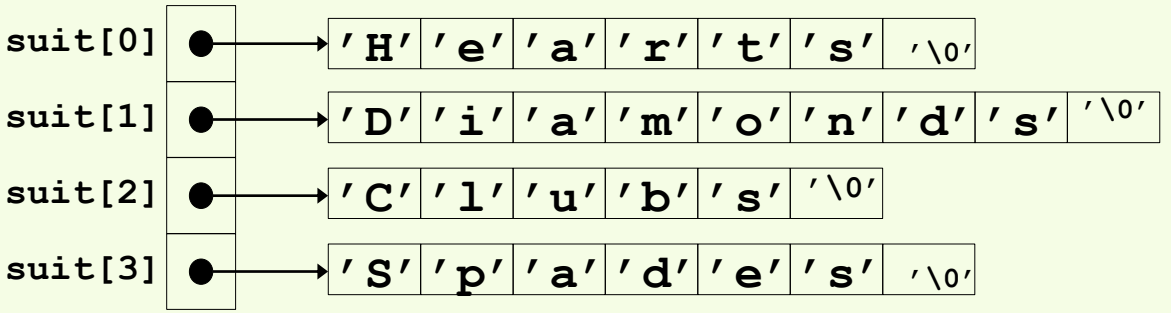
- A pointer stores @
  - int a; int *p=&a; printf(“%p\n”, p);
- Do not leave uninitialized pointer in the program
  - assign it with an @ or null
- & and *
  - *p access to the value!
- Be careful with string:
  - char *s=“SPEIT”; *s='c’;
  - char s[] = "SPEIT"; s[0]='c’;

Pointers and Arrays

- Array name can be thought of as pointer to the first element of the array.
  - int a[10]={0}; *p=a;
  - p: &a[0], *p: a[0];
- Pointer arithmetic
  - Adding an integer to a pointer moves the pointer by a number of positions equal to the integer multiplied by the size of the data type the pointer points to.
  - p+=1, pointer to a[1];
  - printf("%p, %p\n",p,p+1) would have difference of 4 (Bytes)
- Subscripting an array
  - *p : p[0], *(p+1): p[1]

Pointer to pointers

- A pointer to a pointer is a variable that stores the address of another pointer.
  - int a=2; int *p=&a; int **q=&p;
- A pointer to two-dimension array
  - int (*p)[n];
- A pointer array
  - int *p[n];



Pointer and Functions

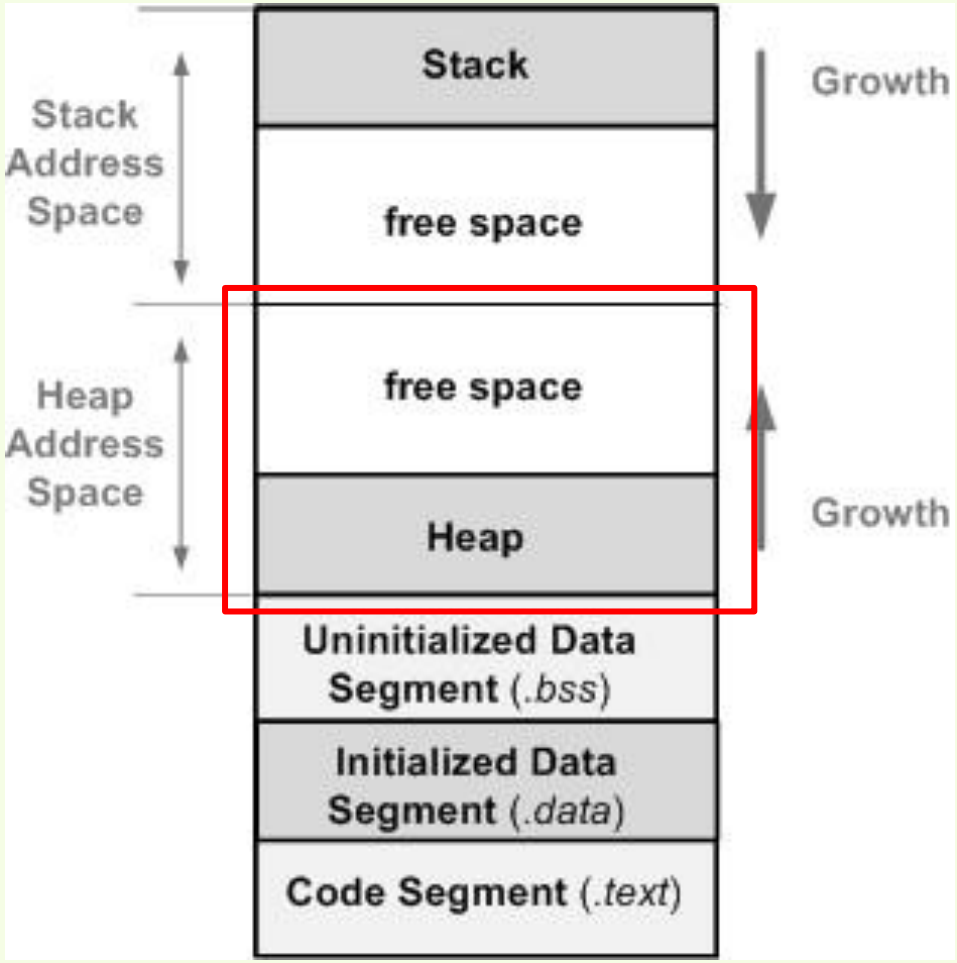
- Call by value
  - Pass the variable value
- Call by reference
  - Pass the variable address / a pointer to it
- Function pointers
  - pointers that point to functions, allowing functions to be passed as arguments to other functions or stored in data structures
  - int max(int, int); int (*p)(int, int); p=max;

```
void swap(int a,int b)
{
    int temp;
    temp=a; a=b; b=temp;
}
int main()
{
    int a=3,b=10;
    swap(a,b);
    printf("%d %d",a,b);
    return 0;
}
```

```
void swap(int *a, int *b)
{
    int *temp;
    temp=a; a=b; b=temp;
}
int main()
{
    int a=3,b=10;
    swap(&a,&b);
    printf("%d %d",a,b);
    return 0;
}
```

Dynamic Memory Allocation

- In C, explicit memory allocation with malloc(), free(), etc.
  - #include <stdlib.h>
  - **`void *malloc(size_t size)`**
  - If successful:
    - Returns a pointer to a memory block of at least size bytes, (typically) aligned to 8-byte boundary.
    - If size == 0, returns NULL
  - If unsuccessful: returns NULL (0).
  - **`void free(void *p)`**
  - Returns the block pointed at by p to pool of available memory
  - p must come from a previous call to malloc()
- Must free!



Dynamic Memory Allocation

- 通常定义变量（或对象），编译器在编译时都可以根据该变量（或对象）的类型知道所需内存空间的大小，从而系统在适当的时候为他们分配确定的存储空间，这种方法称为静态存储分配。
  - 缺点：在大多数情况下会浪费大量的内存空间，在少数情况下，当定义的数组不够大时，可能引起下标越界错误，甚至导致严重后果。
- 有些操作对象只在程序运行时才能确定，这样编译时就无法为他们预定存储空间，只能在程序执行的过程中，系统根据运行时的要求动态地分配或者回收存储空间，这种方法称为动态存储分配。
  - 优点：不需要预先分配存储空间，分配的空间可以根据程序的需要扩大或缩小。

Dynamic Memory Allocation

- 当程序运行到需要一个动态分配的变量或对象时，必须向系统申请取得堆中的一块所需大小的存贮空间，用于存贮该变量或对象。当不再使用该变量或对象时，也就是它的生命结束时，要显式释放它所占用的存贮空间，这样系统就能对该堆空间进行再次分配，做到重复使用有限的资源。

–分配：`void *malloc (unsigned int size)`

  - 返回值是`void *`为"无类型指针"，实际使用时可以进行强制类型转换。

–释放：`free()`

  - `int *pt = (int*)malloc(10*sizeof(int));`
  - `free(pt)`

BASIC ALGORITHM DESIGN

Agenda

- Enumeration
- Greedy
- Divide & Conquer
- Dynamic programing

Enumeration Method (枚举法)

- Enumerate the solutions one by one
- Limited number of solutions
- Find an order to enumerate and check
  - An order to narrow down the search space efficiently

Enumeration Method

- One by one
- Ex: You have 50 Euros to buy fruits. There are 3 kind of fruits:
  - Melon 5 €/piece;
  - Apple 1€ /piece;
  - Orange 1 € /3pieces
- There are totally 100 fruits. Want to taste all kinds of them.
- Output the possible combinations

Naive enumeration

```
int main(){
    int  mellon, apple, orange;
    for (mellon=1; mellon<99; mellon++)
        for ( apple=1; apple<99; apple++) {
            for (orange = 1; orange< 99; orange++)
                if  (mellon+apple+orange == 100) &&  (5 * mellon + 1 * apple + orange /3=<50){
                    printf("mellon: %d, “, mellon);
                    printf("apple: %d, " ,apple);
                    printf("orange: %d\n“, orange);
                }
        }
    return 0;
}
```

循环次数太多，耗时长

Improved enumeration

```
int main(){
    int  mellon, apple, orange;
    for (mellon=1; mellon<10; mellon++)
        for ( apple=1; apple < 50 - 5 * mellon; apple++) {
            orange = 3*(50-5*mellon-apple);
            if  (mellon+apple+orange == 100){
                printf("mellon: %d, “, mellon);
                printf("apple: %d, " ,apple);
                printf("orange: %d\n“, orange);
            }
        }
    return 0;
}
```

有效减少循环次数

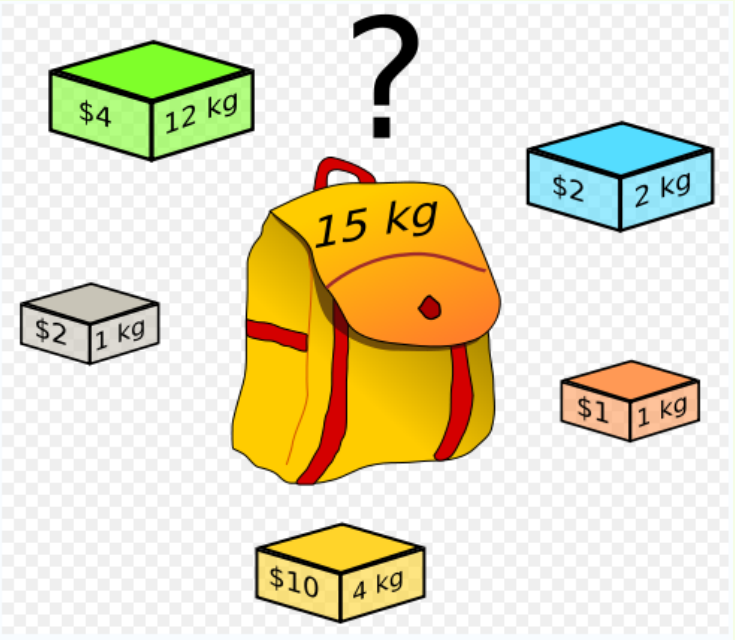
Greedy (贪婪法)

- Targeting at local optimal for each stage
- It is not always guarantee to get the global optimal solution, but it can provide a quite good one with fair cost.

- 贪婪法在每一步都**只做出当前看起来最好的选择**（即局部最优选择），希望这样一步步导出一个全局最优解。但**“局部最优” 的简单叠加，并不总是等于 “全局最优”** 。
- 尽管贪婪法有时会失败，但在许多实际场景中，它仍然非常有用，因为它有两大优势：
  - **计算效率非常高**：它通常非常快，时间复杂度往往是多项式级别（如O(n log n) 或O(n)），因为它只做一次遍历，每次只考虑当前最佳选项，不需要回溯或考虑所有可能性。这对于需要快速得出解决方案的问题至关重要。
  - **解的质量“足够好”**：虽然可能不是数学上最完美的，但它得到的解通常非常接近最优解，在工程和实践上是完全可接受的。这种解被称为 “近似最优解” 。

Knapsack Problem(背包)

- Given the weights and values of N items, in the form of {value, weight} put these items in a knapsack of capacity W to get the maximum total value in the knapsack.
- In Fractional Knapsack, we can break items for maximizing the total value of the knapsack
- In the 0-1 Knapsack problem, we are not allowed to break items. We either take the whole item or don't take it.



- **分数背包问题**：你可以只拿一个物品的**一部分**。
- **0-1背包问题**：每个物品是一个**不可分割的整体**。

Fractional Knapsack

- We have 5 items and a bag limited to 20kg.
- (value, weight) is: (3,6), (5,2), (8,3), (6, 10), (3,8).
- Each item can be broken down!
- What is the load with maximum value?

Fractional Knapsack

- We have 5 items and a bag limited to 20kg.
- (value, weight) is: (3,6), (5,2), (8,3), (6, 10), (3,8).
- Each item can be broken down!
- What is the load with maximum value?
  - Compute the density of each item
    - 0.5, 2.5, 2.666666, 0.6, 0.375
  - Put them from higher density to lower density

- Output: 0.8333,1,1,1,0

Divide and Conquer （分治法）

- Sub problem: Independent

  将一个复杂的大问题**分解**成两个或更多个相同或相似的子问题，直到最后子问题可以简单地直接求解。然后，再将子问题的解**合并**，从而得到原问题的解。

  1.分解：将原问题分解为若干个规模较小、相互独立、**与原问题形式相同**的子问题。

  2.解决：如果子问题规模足够小、可以直接解决，则直接求解。否则，则**递归地**继续分解（即对这个子问题也调用分治法）。

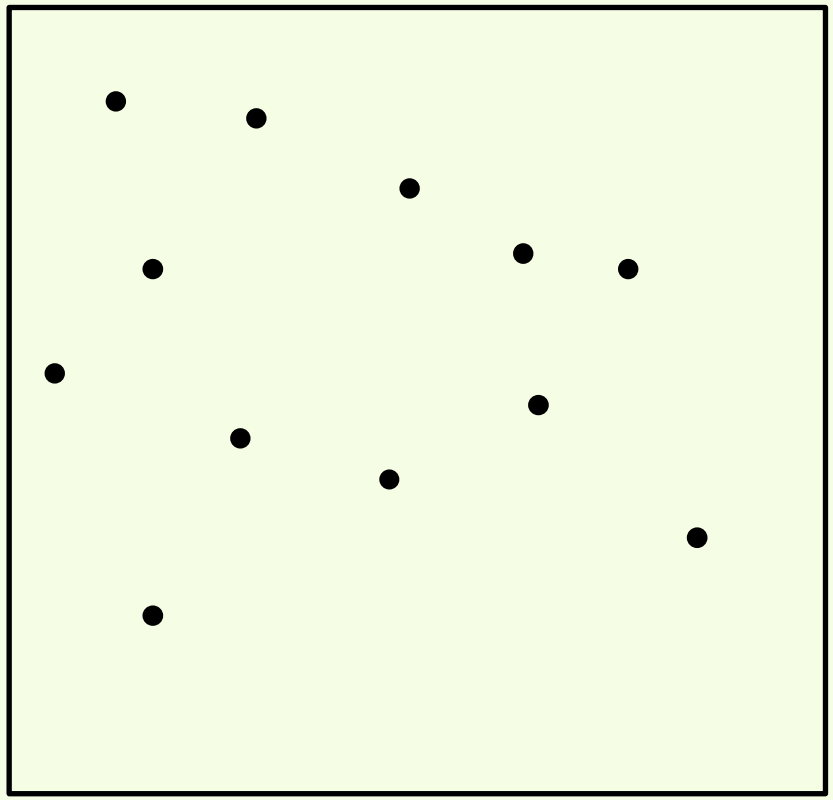
  3.合并：将各个子问题的解**合并**，最终得到原问题的解。

  这个思想非常符合递归的思维方式，因此分治法通常通过**递归**算法来实现。

- Ex: Given N points, you want to find the shortest distance between two points. (最近点对问题)

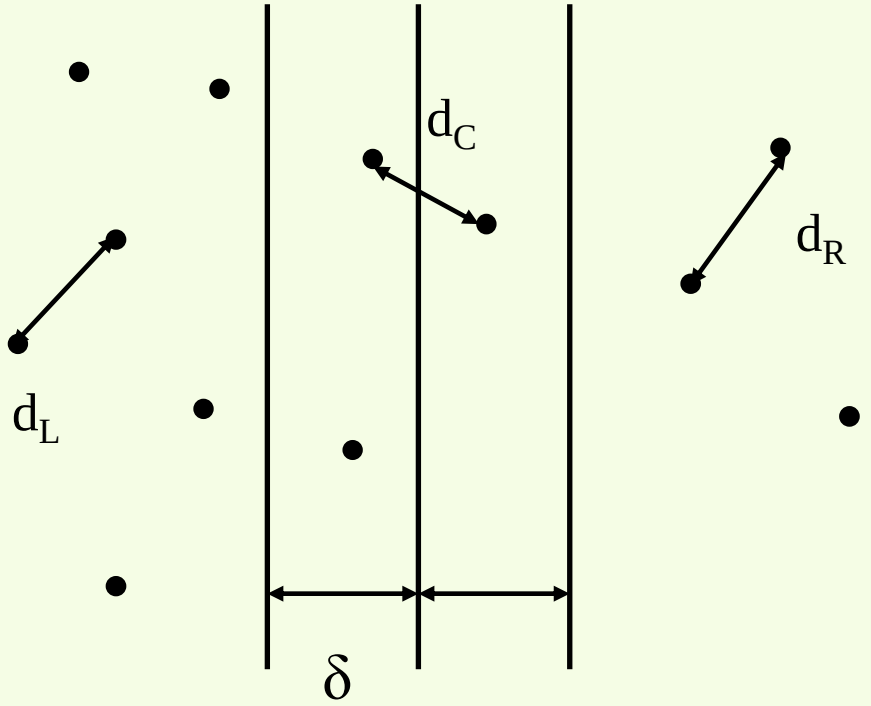
Shortest distance

- Brute-force: Computer all distances of two points and find out the smallest.
- There are N(N-1)/2 distances with N points, the time complexity is $O(N^2)$
- Is there a better one?



Shortest distance

- Sort all points by its x value, divide them into P_L and P_R
- The shortest distance can be found in:
  - d_L, two points in P_L
  - d_R, two points in P_R
    - For d_L and d_R recursive method can be used
  - d_C, one in P_L one in P_R
    - How to compute d_C
      - $\delta = \min(d_L, d_R)$
      - If d_C is larger than δ, then no need to update δ;
      - Only need to computer the points within the band of width δ



Further discussion

- In the worst case, all nodes are in the band δ!!!
- Still do not need to check all the points, can do eliminate by y axe distribution:
  - If the y coordinates of two points is bigger than δ, it is not necessary to compute the distance.

```
for( i= 0; i<n; ++i)
    if  ( p_{j+1}.y -  p_j.y >= δ )
        break;
    else
        if( dist(p_j, p_{j+1}) < δ)
            δ = dist(p_j, p_{j+1});
```

Dynamic Programing （动态规划）

- Store optimal solution for small size problem and reuse them
  - Fibonacci
- Dynamic programming, like the divide-and-conquer method, solves problems by combining the solutions to subproblems.
- But the subproblems are not independent

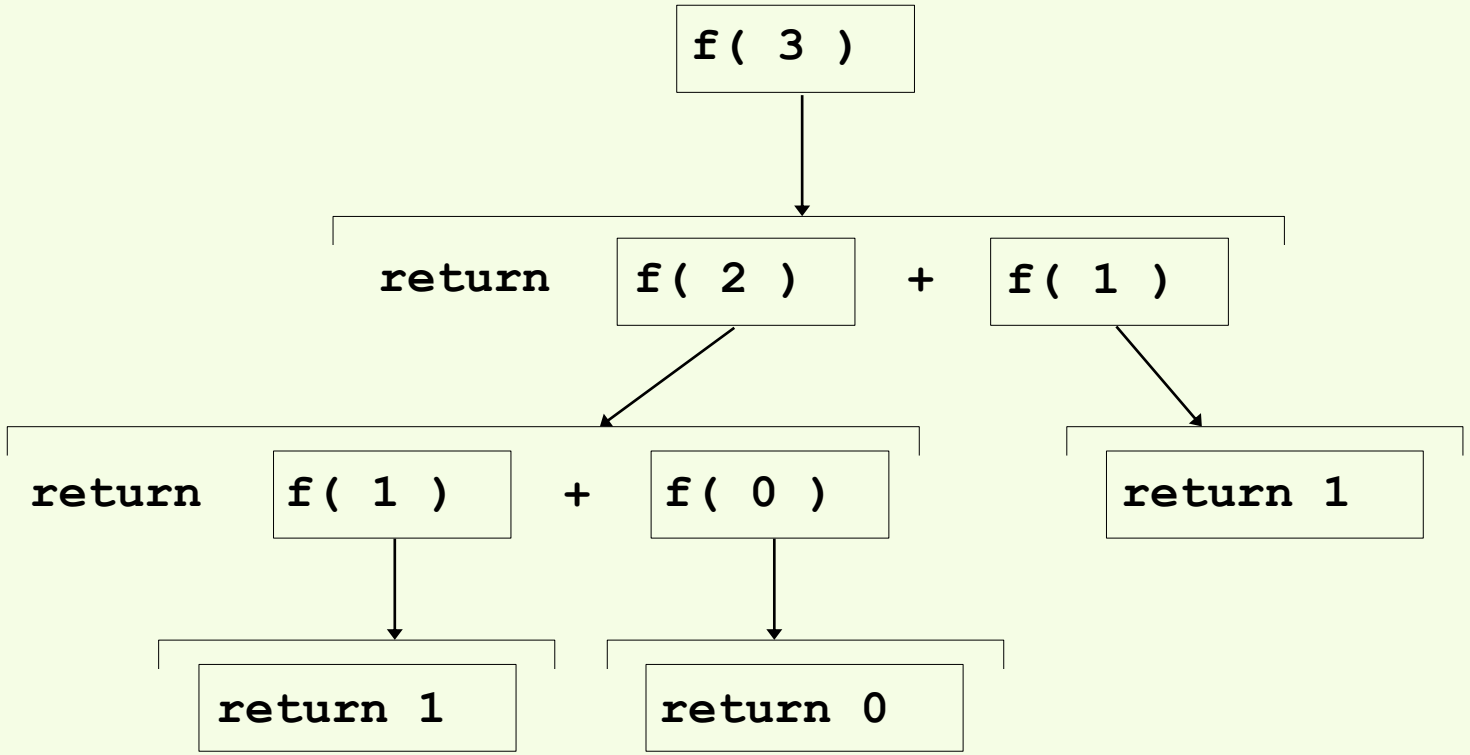
允许子问题相互重叠，有公共的子子问题。

Fibonacci sequence

- The definition of the Fibonacci sequence
  1. procedure f(n)
  2. if n=1 or n=2 then return 1
  3. else return f(n-1)+f(n-2)
- It is concise, easy to write and debug, and, most of all, its abstraction is good.
- But it is far from being efficient.
- Time complexity: $\Theta(\phi^n)$

```
int fib_rec (int n){
      if (n==1 || n==2)
            return 1;
      else
            return fib_rec(n-1) + fib_rec(n-2);
}
```

The Fibonacci Sequence



- **存在大量重复计算**：n 越大，这种重复计算呈指数级增长。
- **树的高度**：从顶部的 fib(n) 到底部的叶子节点 (fib(1), fib(0))，树的高度大约是 n。
- **树的形状**：这棵递归树几乎是一棵满二叉树（虽然不是完全满，但非常接近）。每个节点都会分裂为两个子节点（除了叶子节点）。

What is dynamic programming

- When subproblems share subsubproblems, a divide-and-conquer algorithm does more work than necessary, repeatedly solving the COMMON subsubproblems.
- The dynamic-programming algorithm solves every subsubproblem just once and then saves its answer in a table
- For Fibonacci sequence, an obvious approach is to enumerate the sequence bottom-up starting from f(1) until f(n) is reached. This only takes Θ(n) time and Θ(1) space. A substantial improvement!!!

#1

```
int fib (int n){
    int* array = new int[n];
    array[0] = 0;
    array[1] = 1;
    for (int i = 2; i <= n; i++)
        array[i] = array[i-1] + array[i-2];
    return array[n];
}
```

#2

```
int fib (int n){
    if (n < 2) return n;

    int f0 = 0;
    int f1 = 1;
    int i = 2;
    while (i <= n) {
        f1 = f1 + f0;
        f0 = f1 - f0;
        i++;
    }
    return f1;
}
```

What is dynamic programming

- 动态规划的核心思想是空间换时间，通过存储子问题的解来避免重复计算。适用于具有重叠子问题和最优子结构的问题。
- 重叠子问题
  - 含义：问题可以分解成若干子问题，且这些子问题会被多次重复计算。
  - 动态规划的作用：每个子问题只计算一次，将结果存起来，后续需要时直接查表，从而避免重复计算。
- 最优子结构
  - 含义：一个问题的最优解包含了其子问题的最优解。
  - 为什么重要：只有具备这个性质，我们通过组合子问题的最优解来得到原问题的最优解的策略才是正确的

What is dynamic programming

- 动态规划的两种实现方式：
  - 自顶向下（记忆化搜索）
    - 方法：从原问题开始，像普通递归一样尝试分解问题。但在求解子问题之前，先查表看是否已经计算过。如果没有，再计算并保存结果。
    - 优点：思维模式非常自然，接近于递归。只计算那些实际需要的子问题。
    - 缺点：递归会有额外的函数调用开销。
  - 自底向上（制表法）
    - 方法：从最小的子问题开始，逐步计算并填充表格，直到得到原问题的解。通常使用循环实现。
    - 优点：效率通常更高，避免了递归开销。可以更好地优化空间复杂度。
    - 缺点：有时不太直观，可能需要计算所有子问题，即使有些可能用不到。

Basic steps of dynamic programming

1. Characterize the structure of an optimal solution
2. Recursively define the value of an optimal solution
3. Compute the value of an optimal solution in a bottom-up fashion
4. Construct an optimal solution from computed information (this step can be omitted if only the value is required)

1、刻画一个最优解的结构特征：分析问题，并证明一个问题的最优解包含了其子问题的最优解。换句话说，可以通过组合子问题的最优解来构造原问题的最优解。
2、递归地定义最优解的值：用一个数学方程（称为状态转移方程或递归关系）来定义如何从子问题的最优解得到更大问题的最优解。
3、以自底向上的方式计算最优解的值：使用迭代循环的方式，从最小的子问题开始，逐步填写一张dp表格（即动态规划数组），直到计算出原问题的解。
4、利用计算出的信息构造一个最优解：基于最优解的值，通过回溯 dp 表格，根据状态转移的路径，推断出具体的选择。

0-1 Knapsack

- Each item should be load in one piece (每件物品必须作为一个整体装载)
- 0/1背包问题可以看作是决策一个序列(x_1, x_2, ..., x_n)，对任一变量x_i的决策是决定x_i=1（放入背包）还是x_i=0(不放入背包)。在对x_{i-1}决策后，已确定了(x_1, ..., x_{i-1})，在决策x_i时，问题处于下列两种状态之一：
  - （1）背包容量不足以装入物品i，则x_i=0，背包不增加价值；
  - （2）背包容量可以装入物品i，则x_i=1，背包的价值增加了v_i。
- 这两种情况下背包价值的最大者应该是对xi决策后的背包价值。令$V(i, j)$表示在前$i(1 \le i \le n)$个物品中能够装入容量为j（$1 \le j \le C$）的背包中的物品的最大值，则可以得到如下动态规划函数：

Optimal solution form

$$V(i,j)=\begin{cases}0 & \text{if } i=0 \text{ or } j=0\\ V(i-1,j) & \text{if } j<w_i\\ \max(v_i+V(i-1,j-w_i),V(i-1,j)) & \text{if } j\geq w_i\end{cases}$$

(1) 当物品数量为0或背包容量为0

(2) 当第i个物品重量大于背包容量

(3) 当第i个物品重量不大于背包容量

- (1)表明：把0个物品装入容量为j的背包把前面i个物品装入容量为0的背包，得到的价值均为0。
- (2)表明：如果第i个物品的重量大于背包的容量，则装入前i个物品得到的最大价值和装入前i-1个物品得到的最大价值是相同的，即物品i不能装入背包；
- (3)表明：如果第i个物品的重量小于等于背包的容量，则会有以下两种情况：
  1. 如果把第i个物品装入背包，则背包中物品的价值等于把前i-1个物品装入容量为$j-w_i$的背包中的价值加上第i个物品的价值v_i；
  2. 如果第i个物品没有装入背包，则背包中物品的价值就等于把前i-1个物品装入容量为j的背包中所取得的价值。
  3. 显然，取上二者中价值较大者作为把前i个物品装入容量为j的背包中的最优解。

例如，有5个物品，其重量分别是{2, 2, 6, 5, 4}，价值分别为{6, 3, 5, 4, 6}，背包的容量为10。

根据动态规划函数，用一个(n+1)×(C+1)的二维表V，V[i][j]表示把前i个物品装入容量为j的背包中获得的最大价值。

背包容量		0	1	2	3	4	5	6	7	8	9	10
重量、价值	0											
w_1=2 v_1=6	1											
w_2=2 v_2=3	2											
w_3=6 v_3=5	3											
w_4=5 v_4=4	4											
w_5=4 v_5=6	5											

第一阶段，只装入前1个物品，确定在各种情况下的背包能够得到的最大价值；

第二阶段，只装入前2个物品，确定在各种情况下的背包能够得到的最大价值；

依此类推，直到第n个阶段。最后，$V(n,C)$便是在容量为C的背包中装入n个物品时取得的最大价值。

背包容量		0	1	2	3	4	5	6	7	8	9	10
重量、价值	0	0	0	0	0	0	0	0	0	0	0	0
$w_1=2$ $v_1=6$	1	0	0	6	6	6	6	6	6	6	6	6
$w_2=2$ $v_2=3$	2	0	0	6	6	9	9	9	9	9	9	9
$w_3=6$ $v_3=5$	3	0	0	6	6	9	9	9	9	11	11	14
$w_4=5$ $v_4=4$	4	0	0	6	6	9	9	9	10	11	13	14
$w_5=4$ $v_5=6$	5	0	0	6	6	9	9	12	12	15	15	15

Which are the items in the knapsack?

回溯规则： 从最终状态 dp[i][w] (即 dp[5][10]) 开始，从后往前检查每一个物品 i (从5到1)。

- 如果 dp[i][w] == dp[i-1][w]，说明物品 i 没有被选中。我们只需将 i 减一，w 不变，继续检查前一个物品。
- 如果 dp[i][w] != dp[i-1][w]，说明物品 i 被选中了。我们将物品 i 记录为已选，然后将 w 减去 weight[i]，将 i 减一，继续检查。

背包容量		0	1	2	3	4	5	6	7	8	9	10		
重量、价值	0	0	0	0	0	0	0	0	0	0	0	0	x_1=1	4-2=2
w_1=2 v_1=6	1	0	0	6	6	6	6	6	6	6	6	6	x_2=1	6-2=4
w_2=2 v_2=3	2	0	0	6	6	9	9	9	9	9	9	9	x_3=0	
w_3=6 v_3=5	3	0	0	6	6	9	9	9	9	11	11	14	x_4=0	
w_4=5 v_4=4	4	0	0	6	6	9	9	9	10	11	13	14	x_5=1	10-4=6
w_5=4 v_5=6	5	0	0	6	6	9	9	12	12	15	15	15		

Longest common subsequence (最长公共子序列)

Problem: LCS
Input: Two strings $A=a_1a_2...a_n$ and $B=b_1b_2...b_m$
Output: The longest common subsequence of A and B

ababe

abcabc

给定两个序列 X 和 Y，如果另一个序列 Z 既是 X 的子序列，也是 Y 的子序列，那么 Z 就是 X 和 Y 的公共子序列。

Brute-force method: enumerate all the 2^n subsequence of A, and for each subsequence determine if it is also a subsequence of B in $\Theta(m)$ time. The running time is $\Theta(m*2^n)$

Longest common subsequence

L[i,j]: length of the longest common subsequence of *A[1..i]* and *B[1..j]*

ababe

abcabc

Solution: L[n,m]

$$L[i,j] = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0 \\ L[i-1, j-1] + 1 & \text{if } i > 0, j > 0 \text{ and } a_i = b_j \\ \max\{L[i-1, j], L[i, j-1]\} & \text{if } i > 0, j > 0 \text{ and } a_i \neq b_j \end{cases}$$

(1) 当A或B序列长度为0

(2) 当前字符匹配

(3) 当前字符不匹配

- (1)表明：当A或B序列长度为0，得到的LCS的长度为0。
- (2)表明：如果当前字符匹配(a_i == b_j)，它必然是公共子序列的一部分，所以LCS的长度就是“去掉这个字符的两个子序列的LCS长度 + 1”；
- (3)表明：如果当前字符不匹配(a_i != b_j)，那么LCS的长度只能继承自之前的最佳结果。它只能是 “X 的前i-1个字符和 Y 的前j个字符的LCS长度” 或者 “X 的前i个字符和 Y 的前j-1个字符的LCS长度” 中的最大值。

LCS

$$L[i,j] = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0 \quad (1) \\ L[i-1,j-1]+1 & \text{if } i>0, j>0 \text{ and } a_i = b_j \quad (2) \\ \max\{L[i-1,j], L[i,j-1]\} & \text{if } i>0, j>0 \text{ and } a_i \neq b_j \quad (3) \end{cases}$$

	0	a	b	c	a	b	c
0							
a							
b							
a							
b							
e							

填充dp表步骤：

LCS

$$L[i,j] = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0 \quad (1) \\ L[i-1,j-1]+1 & \text{if } i>0, j>0 \text{ and } a_i = b_j \quad (2) \\ \max\{L[i-1,j], L[i,j-1]\} & \text{if } i>0, j>0 \text{ and } a_i \neq b_j \quad (3) \end{cases}$$

	0	a	b	c	a	b	c
0	0	0	0	0	0	0	0
a	0						
b	0						
a	0						
b	0						
e	0						

填充dp表步骤：

- 根据(1), L均为0；

LCS

$$L[i,j] = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0 \quad (1) \\ L[i-1,j-1]+1 & \text{if } i>0, j>0 \text{ and } a_i = b_j \quad (2) \\ \max\{L[i-1,j], L[i,j-1]\} & \text{if } i>0, j>0 \text{ and } a_i \neq b_j \quad (3) \end{cases}$$

	0	a	b	c	a	b	c
0	0	0	0	0	0	0	0
a	0	1					
b	0						
a	0						
b	0						
e	0						

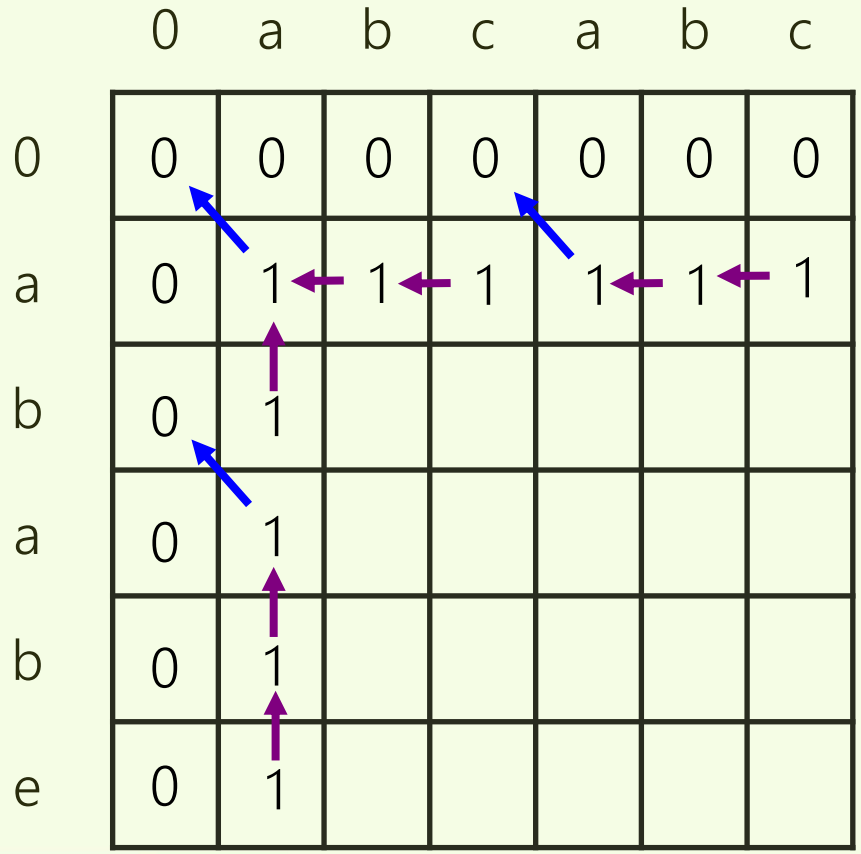
填充dp表步骤：

- 根据(1), L均为0；
- 根据(2), L[a,a]=L[0,0]+1=1；

LCS

$$L[i,j] = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0 \quad (1) \\ L[i-1,j-1]+1 & \text{if } i>0, j>0 \text{ and } a_i = b_j \quad (2) \\ \max\{L[i-1,j], L[i,j-1]\} & \text{if } i>0, j>0 \text{ and } a_i \neq b_j \quad (3) \end{cases}$$

蓝线来自公式(2)，
紫线来自公式(3)



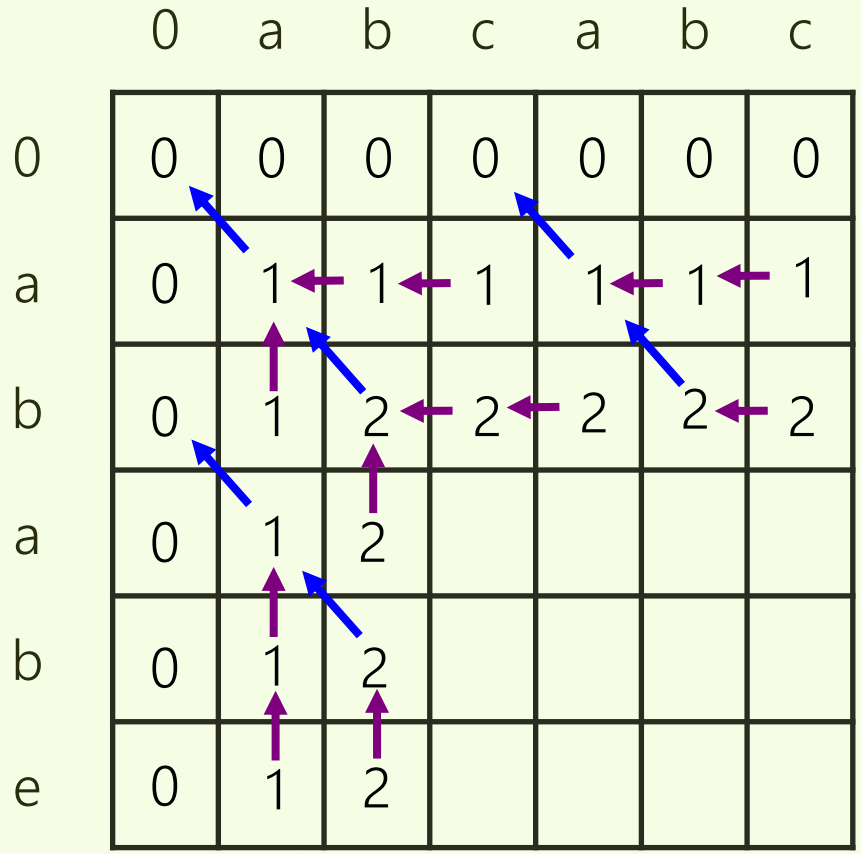
填充dp表步骤：

- 根据(1), L[0,j]和L[i,0]均为0；
- 根据(2), L[a,a]=L[0,0]+1=1；
- 根据(3), L[a,b]=L[a,a]=1，该行以此类推，
  L[b,a]=L[a,a]=1，该列以此类推；

LCS

$$L[i,j] = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0 \quad (1) \\ L[i-1,j-1]+1 & \text{if } i>0, j>0 \text{ and } a_i = b_j \quad (2) \\ \max\{L[i-1,j], L[i,j-1]\} & \text{if } i>0, j>0 \text{ and } a_i \neq b_j \quad (3) \end{cases}$$

蓝线来自公式(2)，
紫线来自公式(3)



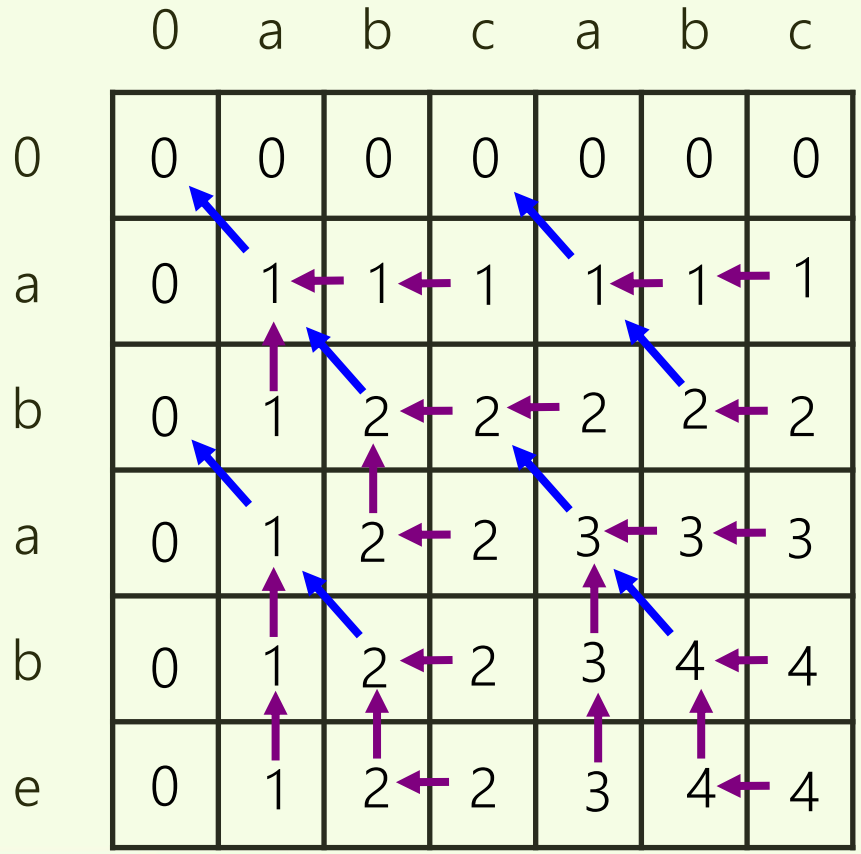
填充dp表步骤：

- 根据(1), L[0,j]和L[i,0]均为0；
- 根据(2), L[a,a]=L[0,0]+1=1；
- 根据(3), L[a,b]=L[a,a]=1，该行以此类推，
  L[b,a]=L[a,a]=1，该列以此类推；
- 根据(2), L[b,b]=L[a,a]+1=2;
- 根据(3), L[b,c]=L[b,b]=2，该行以此类推，
  L[a,b]=L[b,b]=2，该列以此类推；

LCS

$$L[i,j] = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0 \quad (1) \\ L[i-1,j-1]+1 & \text{if } i>0, j>0 \text{ and } a_i = b_j \quad (2) \\ \max\{L[i-1,j], L[i,j-1]\} & \text{if } i>0, j>0 \text{ and } a_i \neq b_j \quad (3) \end{cases}$$

蓝线来自公式(2)，
紫线来自公式(3)



填充dp表步骤：

- 根据(1), L[0,j]和L[i,0]均为0；
- 根据(2), L[a,a]=L[0,0]+1=1；
- 根据(3), L[a,b]=L[a,a]=1，该行以此类推，
  L[b,a]=L[a,a]=1，该列以此类推；
- 根据(2), L[b,b]=L[a,a]+1=2;
- 根据(3), L[b,c]=L[b,b]=2，该行以此类推，
  L[a,b]=L[b,b]=2，该列以此类推；
- 以此类推，补完表格。

LCS

Algorithm 7.1 LCS
Input: Two strings A and B of length n and m over Σ.
Output: The length of the longest common subsequence of A, B.

```
1.  for i ← 0 to n do L[i,0] ← 0 end for
2.  for j ← 0 to m do L[0,j] ← 0 end for
3.  for i ← 1 to n
4.    for j ← 1 to m
5.      if a_i = b_j then L[i,j] ← L[i−1,j−1] + 1
6.      else L[i,j] ← max{L[i−1,j], L[i,j−1]}
7.      end if
8.    end for
9.  end for
10. return L[n,m]
```

Time: Θ(mn)
Space: Θ(max{m,n})

Summary

- Analyze the problem before resolve it.
- Find out the application range of each classic algorithm